



Political Patricians (patricians)

Um censor romano está a monitorizar N patrícios com lealdades políticas duvidosas. Cada patrício pertence a **exatamente uma** facção política secreta e cada facção tem um ou mais patrícios. O censor quer determinar quais patrícios pertencem a quais facções.

Para isso, o censor organizará repetidamente banquetes nos quais todos os N patrícios serão convidados. Em cada banquete, os patrícios podem sentar-se em mesas diferentes e o censor controla a disposição dos lugares. Um patrício é considerado *solitário* se não houver nenhum outro patrício da mesma facção política sentado à mesma mesa.

Como mestre de cerimónias, o censor saberá após cada banquete quais os patrícios que ficaram solitários. A tua tarefa é ajudar o censor a determinar as lealdades políticas dos patrícios sem organizar demasiados banquetes.

Implementação

Tens de submeter um único ficheiro de código .cpp.



Entre os ficheiros anexos deste problema encontra-se um template `patricians.cpp` com um exemplo de implementação.

Tens de implementar a seguinte função:

C++	<code>vector<int> find_factions(int N)</code>
-----	---

- O inteiro N representa o número de patrícios. Os patrícios são numerados de 0 a $N - 1$.
- A função deve devolver um array P de N números inteiros representando as facções políticas dos patrícios. $P[i]$ deve ser igual a $P[j]$ se e só se os patrícios i e j pertencerem à mesma facção política. Não existem outras restrições sobre os valores dos inteiros de P .

O teu programa pode chamar a seguinte função, que já está definida no avaliador:

C++	<code>vector<bool> organize_banquet(const vector<int>& Q)</code>
-----	--

- O array Q de N números inteiros representa a disposição dos lugares: o patrício i sentar-se-á à mesa $Q[i]$. Não há restrições quanto aos valores dos inteiros de Q : as mesas podem receber qualquer numeração, e a única informação relevante é que, se $Q[i] = Q[j]$, então o patrício i e o patrício j estarão sentados na mesma mesa.
- A função devolverá um array A de N booleanos. $A[i]$ será verdadeiro se e só se o patrício i estiver sozinho no banquete.

O avaliador **não é adaptativo** e cada envio é avaliado com o mesmo conjunto de casos de teste. Dito de outro modo, as facções políticas de cada patrício são fixadas antecipadamente em cada caso de teste e não dependem das chamadas a `organize_banquet` feitas pelo teu programa.

Avaliador de Exemplo

Entre os ficheiros anexos deste problema, encontrarás uma versão simplificada do avaliador usado numa submissão real, que poderás usar para testar as soluções localmente. O avaliador lê dados de `stdin`, chama a função `find_factions` e escreve de volta em `stdout` usando o seguinte formato.

O input é constituído por 2 linhas, contendo:

- Linha 1: o inteiro N .
- Linha 2: N inteiros que descrevem a facção política de cada patrício.

O output é constituído por uma linha, contendo a pontuação obtida pela tua solução no caso de teste. Se a pontuação for 0, será imprimida uma breve mensagem para o `stderr` com o motivo (facções incorretas dadas como resposta, consulta inválida a `organize_banquet` ou consultas em excesso).

Restrições

- $4 \leq N \leq 2000$.

Podes chamar `organize_banquet` no máximo 30000 vezes.

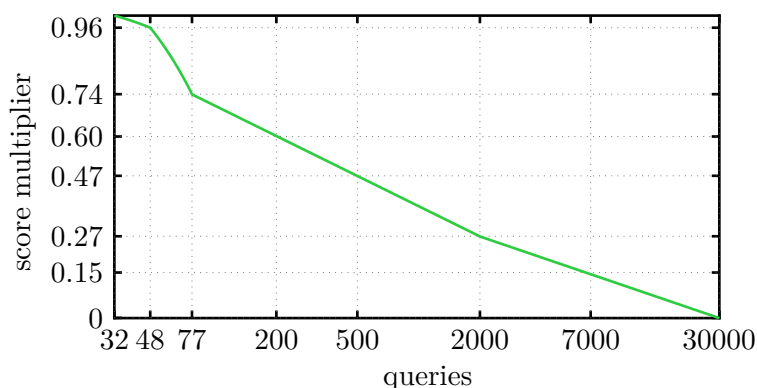
Pontuação

O teu programa será testado com vários casos de teste agrupados em subtarefas. O multiplicador de pontuação associado a uma subtarefa será o mínimo dos multiplicadores de pontuação obtidos em cada um dos casos de teste dessa subtarefa.

- **Subtarefa 0 [0 pontos]:** Casos de exemplo.
- **Subtarefa 1 [19 pontos]:** Existem no máximo duas facções políticas
- **Subtarefa 2 [29 pontos]:** Cada facção política contém exatamente dois patrícios.
- **Subtarefa 3 [52 pontos]:** Nenhuma restrição adicional.

O multiplicador de pontuação obtido num caso de teste é determinado da seguinte maneira (uma consulta é uma chamada a `organize_banquet`).

Número de consultas	Multiplicador de pontuação
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



A pontuação numa subtarefa é o mínimo multiplicador de pontuação obtido num dos seus casos de teste, multiplicado pela pontuação base da subtarefa.

Exemplos

Avaliador	Solução
<code>find_factions(5)</code>	
	<code>organize_banquet([0,0,1,1,2])</code>
<code>return [1,1,1,1,1]</code>	
	<code>organize_banquet([0,0,0,1,1])</code>
<code>return [0,1,0,1,1]</code>	
	<code>organize_banquet([0,1,2,1,1])</code>
<code>return [1,0,1,0,1]</code>	
	<code>return [0,1,0,1,2]</code>

Avaliador	Solução
<code>find_factions(4)</code>	
	<code>organize_banquet([1,1,0,0])</code>
<code>return [1,1,1,1]</code>	
	<code>organize_banquet([1,1,1,2])</code>
<code>return [0,1,0,1]</code>	
	<code>organize_banquet([0,1,1,1])</code>
<code>return [1,0,1,0]</code>	
	<code>return [0,1,0,1]</code>

Explicação

No primeiro exemplo, os patrícios 0 e 2 estão numa facção política, os patrícios 1 e 3 estão noutra diferente e o patrício 4 está na sua própria facção. No primeiro banquete, todos os patrícios estão sozinhos. No segundo banquete, os patrícios 0 e 2 não estão sozinhos porque estão sentados juntos. No terceiro banquete, os patrícios 1 e 3 não estão sozinhos porque estão sentados juntos.